



COMPUTER PROGRAMMING QUESTION BANK

MODULE-1: INTRODUCTION

SHORT ANSWER TYPE QUESTIONS:

✓ 1. Define software and programming.

Ans:

SOFTWARE:

It is basically a set of instructions or commands that tell a computer to do a specific task that serves its users.

Examples are MS-Word, MS-Excel, PowerPoint and Web Browser.

PROGRAMMING:

Programming is the process of designing, writing, testing, and maintaining a set of instructions (called code) that a computer follows to perform a specific task.

These instructions are written using programming languages like C, Python, Java, etc.

✓ 2. What is type conversion? Give one example.

Ans:

Type Conversion in Python refers to the process of converting one data type into another. This is often necessary when performing operations that require data to be in a specific format.

- Convert the data type of an object to another.

Syntax: **datatype(object)**

- Changes can be stored in same variable or in different variable.

Example:

```
a=23.65
b=int(a)
print(type(b))

<class 'int'>
```

✓ 3. Define syntax in python.



Ans: The set of guidelines that specify the organization and sequencing of statements in a programming language are referred to as “syntax”. The python syntax is a collection of guidelines that define how the language can be expressed and used. When writing code, Python syntax comprises keywords, operators, expressions, statements.

✓ **4. Define variable with an example in Python.**

Ans: A variable in Python is a symbolic name that acts as a reference or container for storing data values. Values assigned to variables using an assignment operator ‘=’. Variables can be named alphanumerically. However, the first letter must start with an alphabet (lowercase, uppercase).

Example: abc=12, a_bc=13, ab1=23.....

✓ **5. List any four arithmetic operators in Python.**

Ans: Arithmetic operators are used to perform mathematical operations between two operands.

SYMBOL	OPERATION
+	Addition
-	Subtraction
*	Multiplication
/	Division
**	Exponent
//	Double division

✓ **6. What is the difference between = and == operators?**

Ans: In Python, = and == are fundamentally different operators with distinct purposes:

1. = (Assignment Operator):

- Used to assign a value to a variable.
- It does not compare values; it simply stores the value on the right-hand side into the variable on the left-hand side.

Example:

a = 5 # Assigns the value 5 to the variable a

b = "Hello world" # Assigns the string "Hello" to the variable b



2. == (Equality Operator):

- Used to compare two values for equality.
- Returns True if the values on both sides are equal, otherwise returns False.

Example:

```
a = 10
```

```
print(a == 10) # True, because x is equal to 10
```

```
print(a == 5) # False, because x is not equal to 5
```

7. What is a data type in programming.

Ans: The data types are used to define the type of a variable. It represents the type of data we are going to store in a variable and determines what operations can be done on it.

Syntax: type(variable name)

8. Write short notes on python input-output functions.

Ans: With the print() function, we can display output in various formats, while the input() function enables interaction with users by gathering input during program execution.

Taking input in Python:

Python's input() function is used to take user input. By default, it returns the user input in form of a string.

Example:

```
name = input("Enter your name: ")  
print(name)
```

```
Enter your name: Hello world  
Hello world
```

Output Function:

We use the print() function or print keyword to output data to the standard output device (screen). This function prints the object/string written in function.



```
print("Hello, World!")
```

Hello, World!

LONG ANSWER TYPE QUESTIONS:

1. Discuss why Python is widely used compared to other programming languages.

Ans: The primary factors cited by Python users are:

- 1. Software quality:** Python code is designed to be **readable**, and hence, **reusable** and **maintainable** much more than traditional scripting languages. In addition, Python has deep support for more advanced software reuse mechanisms, such as object-oriented programming (OOP).
- 2. Developer productivity:** Python boosts developer productivity many times beyond compiled or statically typed languages such as C, C++, and Java. Python code is typically one-third to one-fifth the size of equivalent C++ or Java code.
- 3. Program portability:** Most Python programs run unchanged on all major computer platforms. Porting Python code between Linux and Windows, for example, is usually just a matter of copying a script's code between machines.
- 4. Support libraries:** Python comes with a large collection of prebuilt and portable functionality, known as the **standard library**. Python can be extended with both homegrown libraries and a vast collection of third-party application support software.
- 5. Component integration:** Python scripts can easily communicate with other parts of an application, using a variety of integration mechanisms. Such integrations allow Python to be used as a product customization and extension tool.
- 6. Enjoyment:** Because of Python's ease of use and built-in toolset, it can make the act of programming more pleasure than chore.

✓ 2. Explain variable declaration rules in Python with suitable examples.

Ans:

- A variable in Python is a symbolic name that acts as a reference or container for storing data values.
- Values assigned to variables using an assignment operator '='.



Example: a=12

- Variable name should be short and descriptive.
- Avoid using variable names that clash with inbuilt functions.

Example:

```
if=23  
print(if)
```

SyntaxError: invalid syntax

- Variables can be named alphanumerically.

Example: abc=12, a_bc=13, ab1=23.....

- However, the first letter must start with an alphabet (lowercase, uppercase).

Example: Abc=10, a=10

- Underscore (_) is the only special character used in variable declaration.

Example: even_odd=45

- Use of any other special character will throw an error.

Example:

```
a&c=15  
print(a&c)
```

Output:

SyntaxError: cannot assign to expression here. Maybe you meant '==' instead of '='?

- Variable names should not begin or end with underscore even though both are allowed

Example: _age=55

ge_=55

- Assigning multiple values to multiple variables:



```
abc,cde,efg=10,20,30
print("abc=",abc)
print("cde=",cde)
print("efg=",efg)
```

Output:

```
abc= 10
cde= 20
efg= 30
```

3. Explain briefly about data types in python.

✓ **Ans:** A data type defines the kind of value an object can hold and the operations that can be performed on it. Everything in Python is an object, and every object has a data type.

Python's data types are broadly categorized by

Object Type	Description	Example
Fundamental or Build-in Data types or Data Structures		
1. int	We can use to represent the whole/integral Numbers	A=26,b=10, C=-12, d=-26
2. float	We can use to represent the decimal/floating point numbers	A=26.6, b=-26.2
3. complex	We can use to represent the complex numbers	C=26+26j
4. bool	We can use to represent the logical values(Only allowed values are True and False)	X=True, y= False
5. str	To represent sequence of Characters	A="MRV", v="hello"
Composite Data Types or Data Structures		
6. range	To represent a range of values	r=range(26) r1=range(1,26) r2=range(1,2,3)
7. list	To represent an ordered collection of objects	L1=[1,2,3,4,5]



8. tuple	To represent an ordered collections of objects	t=(1,2,3,4,5)
9. set	To represent an unordered collection of unique Objects	S={1,2,3,4,5}
10. dict	To represent a group of key value pairs	d={1:'abc',2:'def',3:'xyz'}
11. None	None means Nothing or No value associated.	a=None

4/ Explain type conversions in detail.

Ans: Type Conversion in Python refers to the process of converting one data type into another. This is often necessary when performing operations that require data to be in a specific format.

Coercing object to new data type:

- Convert the data type of an object to another

Syntax: datatype(object)

- Changes can be stored in same variable or in different variable

```
Employee_name="Ram"  
Age=55  
Height=150.6
```

Coercing the data type of an object

```
height=150.6  
ht= int(height)  
print(type(ht))
```

<class 'int'>

```
height=int(height)  
print(type(ht))
```

<class 'int'>

- Only few coercions are accepted
- Consider the variable 'Salary_tier' which is of string data type
- 'Salary_tier' contains an integer enclosed between single quotes.



```
Salary_tier='1'
```

Coercing the data type of an object

```
Salary_tier='1'  
print(type(Salary_tier))  
  
<class 'str'>  
  
Salary_tier= int(Salary_tier)  
print(type(Salary_tier))  
  
<class 'int'>
```

- However, if the value enclosed within the quotes is a string then conversions will not be possible.

Example:

```
Employee_name='Ram'  
a=float(Employee_name)
```

Output:

```
1 Employee_name='Ram'  
----> 2 a=float(Employee_name)  
  
ValueError: could not convert string to float: 'Ram'
```

5. Define operator and classify its types. Write a short note on logical operators in Python with examples.

Ans: Operators are special symbols that help in carrying out an assignment operation or arithmetic or logical computation. Value that the operator operates on is called operand

Example: a+b # here a and b are operands whereas '+' indicates operator.

Types of Operators:

1. Arithmetic Operator
2. Logical Operators
3. Assignment Operator
4. Comparison operator
5. Bitwise operator



❖ LOGICAL OPERATORS:

Logical operators are used when operands are conditional statements and returns Boolean value. In python, logical operators are designed to work with Boolean values. These operators include **and**, **or**, and **not**, each serving a specific purpose in logical operations.

1. **and:** Returns **True** if both conditions are **True**. If either condition is **False**, the result is **False**.

Example:

```
x=4
y=10
print((x<5) and (y>7))
```

True

```
x=4
y=10
print((x<5) and (y<7))
```

False

2. **or:** Returns **True** if at least one condition is **True**. If both conditions are **False**, the result is **False**.

Example:

```
x=4
y=10
print((x<5) or (y<7))
```

True

```
x=4
y=10
print((x>5) or (y<7))
```

False

3. **not:** Reverses the Boolean value of the condition. If the condition is **True**, it returns **False**, and vice versa.

Example:

```
x=False
print(not x)
```

True

```
x=True
print(not x)
```

False

✓ **5. Explain briefly about Bitwise operators in python with examples.**



Ans: Bitwise operators in Python are used to perform operations on binary representations of integers. These operators work at the bit level, manipulating individual bits of the operands.

BITWISE AND (&):

- **Bitwise And(&)** compares each bit of two numbers and returns 1 if both bits are 1, otherwise 0.

Example:

```
a=9  
b=7  
print(a & b)
```

1

BITWISE OR (|)

- **Bitwise Or(|)** compares each bit of two numbers and returns 1 if at least one of the bits is 1.

Example:

```
a=9  
b=7  
print(a | b)
```

15

BITWISE XOR (^)

- **Bitwise XOR** Compares each bit of two numbers and returns 1 if the bits are different, otherwise 0.

Example:

```
a=9  
b=7  
print(a ^ b)
```

14

BITWISE NOT (~)



- **Bitwise NOT** inverts all the bits of the number (1 becomes 0, and 0 becomes 1). It also changes the sign of the number due to two's complement representation.

Example:

```
a=9  
print(~a)  
  
-10
```

BITWISE LEFT SHIFT (<<):

- **Bitwise Left Shift** shifts the bits of the number to the left by the specified number of positions, filling with 0s on the right.

Example:

```
a=9  
print(a<<2)  
  
36
```

BITWISE RIGHT SHIFT (>>):

- **Bitwise Right Shift** shifts the bits of the number to the right by the specified number of positions, discarding bits shifted out.

Example:

```
a=9  
print(a>>1)  
  
4
```

7 ✓ Explain briefly about Assignment operators in python with examples.

Ans:

Assignment operators in Python are used to assign values to variables and perform operations while updating the variable in place. They combine arithmetic or bitwise operations with assignment, making code shorter and more efficient.



1. = (Simple Assignment): Assigns the value on the right to the variable on the left.

- **Example:**

```
x = 10  
print(x)
```

Output: 10

2. += (Add and Assign): Adds the right operand to the left operand and assigns the result.

- **Example:**

```
x = 5  
x += 3  
print(x)
```

Output: 8

3. -= (Subtract and Assign): Subtracts the right operand from the left operand and assigns the result.

- **Example:**

```
x = 10  
x -= 4  
print(x)
```

Output: 6

4. *= (Multiply and Assign): Multiplies the left operand by the right operand and assigns the result.

- **Example:**

```
x = 7  
x *= 2  
print(x)
```

Output: 14

5. /= (Divide and Assign): Divides the left operand by the right operand and assigns the result.

- **Example:**

```
x = 20
```



```
x /= 5  
print(x)
```

Output: 4.0

6. %= (Modulo and Assign): Performs modulo operation and assigns the result.

- **Example:**

```
x = 17  
x %= 5  
print(x)
```

Output: 2

7. **= (Exponent and Assign): Raises the left operand to the power of the right operand and assigns the result.

- **Example:**

```
x = 3  
x **= 4  
print(x)
```

Output: 81

8. //= (Floor Divide and Assign): Performs floor division and assigns the result.

- **Example:**

```
x = 17  
x //= 3  
print(x)
```

Output: 5

8. ✓ Explain briefly about Arithmetic operators in python with examples.

Ans:

Arithmetic operators in Python are symbols used to perform basic mathematical operations like addition, subtraction, multiplication, division, and more. They work on numerical values and return results accordingly.

1. Addition (+): Adds two numbers.

- **Example:**

```
a = 5
```



```
b = 3
```

```
print(a + b)
```

Output: 8

2. Subtraction (-): Subtracts the right operand from the left operand.

- **Example:**

```
a = 10
```

```
b = 4
```

```
print(a - b)
```

Output: 6

3. Multiplication (*): Multiplies two numbers.

- **Example:**

```
a = 7
```

```
b = 2
```

```
print(a * b)
```

Output: 14

4. Division (/): Divides the left operand by the right operand (returns float).

- **Example:**

```
a = 20
```

```
b = 5
```

```
print(a / b)
```

Output: 4.0

5. Floor Division (//): Divides and returns the integer (floor) value.

- **Example:**

```
a = 17
```

```
b = 3
```

```
print(a // b)
```

Output: 5

6. Modulus (%): Returns the remainder of division.

- **Example:**

```
a = 17
```



```
b = 5
```

```
print(a % b)
```

Output: 2

7. Exponentiation ():** Raises the left operand to the power of the right operand.

- **Example:**

```
a = 3
```

```
b = 4
```

```
print(a ** b)
```

Output: 81

9/ Explain comparison operators in Python with examples.

Ans: Comparison operators, also known as relational operators, are used to compare two values. They return a Boolean value: True if the condition is met, and False otherwise. These operators are commonly used in decision-making and loops.

1. Equal to (==)

The equality operator checks if two values are exactly the same.

Example:

```
a=5
b=4
c=5
print(a==b)
print(a==c)
```

Output:

```
False
True
```

2. Not Equal to (!=):

The inequality operator checks if two values are not equal.

Example:



```
a=5  
b=4  
c=5  
print(a!=b)  
print(a!=c)
```

Output:

```
True  
False
```

3. Greater Than Operator (>):

This operator checks if the left operand is larger than the right.

Example:

```
a=5  
b=4  
print(a>b)
```

Output:

```
True
```

4. Less Than Operator (<):

It checks if the left operand is smaller than the right.

Example:

```
a=5  
b=4  
print(a<b)
```

Output:

```
False
```

5. Greater Than or Equal To Operator (>=)

It checks if the left operand is greater than or equal to the right.



Example:

```
a=5  
b=4  
print(a>=b)
```

Output: True

6. Less Than or Equal To Operator (<=)

It checks if the left operand is less than or equal to the right.

Example:

```
a=5  
b=4  
print(a<=b)
```

Output: False

10.a) Explain Operator Precedence in python

b) Simplify Expression A = 7-2*(27/32)+4 using Operator Precedence.**

Ans:

a) Operator Precedence in Python:

Decreasing order of precedence	Operation
Parentheses	()
Exponent	**
Division	/
Multiplication	*
Addition and subtraction	+, -
Bitwise AND	&
Bitwise OR	
Relational/ comparison operators	==, !=, >, >=, <=



Logical NOT	not
Logical AND	and
Logical OR	or

b) **Simplify Expression $A = 7 - 2 * (27 / 3 ** 2) + 4$ using Operator Precedence.**

Expression:

$$A = 7 - 2 \times \left(\frac{27}{3^2} \right) + 4$$

Step 1: Evaluate exponentiation 3^2

$$3^2 = 9$$

Expression becomes:

$$7 - 2 \times \left(\frac{27}{9} \right) + 4$$

Step 2: Perform division $\frac{27}{9}$

$$\frac{27}{9} = 3$$

Expression becomes:

$$7 - 2 \times 3 + 4$$

Step 3: Perform multiplication 2×3

$$2 \times 3 = 6$$

Expression becomes:

$$7 - 6 + 4$$

Step 4: Perform subtraction and addition from left to right

$$7 - 6 = 1$$

$$1 + 4 = 5$$

Final value:

$$A = 5$$



MODULE-II: CONTROL STRUCTURES AND FUNCTIONS

SHORT ANSWER TYPE QUESTIONS:

1. Write the syntax of an if statement in Python.

Ans: An if statement is used as a form of conditional programming; something you probably do every day in the real world.

In its most basic form, the if statement is

if <condition-evaluating-to-boolean>:

statement

It is important that the condition must evaluate to True or False. If the condition is True, then it will execute the indented statement.

2. What is the difference between local and global variables?

Ans:

Comparison	Local Variable	Global Variable
Definition	Local variables are declared within the functions.	Global variables are declared outside the functions.
Scope	Accessible inside the function.	Accessible throughout the code.
Parameter Passing	For local variables, parameter passing is necessary.	For global variables, parameter passing is not necessary.
Naming Restrictions	Different variables with the same name can be declared in different functions.	Global variables should be uniquely named i.e. many variables with the same name cannot exist.

3. Define return function with an example.

Ans: In Python, whenever a return statement is encountered within a function then that function will terminate and return any values following the return keyword.

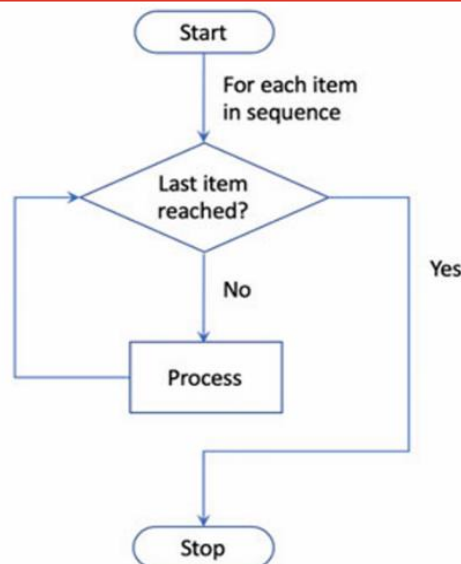
For example, the following defines a simple function that squares whatever value has been passed to it: When we call this function, it will multiple whatever it is given by itself and then return that value.



```
def square(n):  
    return n * n  
  
square(45)  
  
2025
```

4. What is a for loop in Python?

A for loop in Python is used to repeat a block of code for each item in a sequence. It is mainly used when the number of repetitions is known.



5. Define the terms Parameters and Arguments.

Ans:

Parameter: A parameter is a variable defined as part of the function header and is used to make data available within the function itself.

Argument: An argument is the actual value or data passed into the function when it is called. The data will be held within the parameters.

6. What is the use of the elif keyword?

Ans: In some cases, there may be several conditions you want to test, with each condition being tested if the previous one failed. This else-if scenario is supported in



Python by the elif element of an if statement. The elif element of an if statement follows the if part and comes before any (optional) else part.

7. Define a function in Python.

Ans: In Python, Functions are groups of related statements that can be called together, that typically perform a specific task, and which may or may not take a set of parameters or return a value. Functions can be defined in one place and called or invoked in another. This helps to make code more easier to understand.

8. What is `isdigit()` string method?

Ans: The `isdigit()` method in Python is a built-in string method that checks whether all the characters in a string are digits. It returns **True** if every character in the string is a **digit** (0-9). However, it also returns **True** for certain **Unicode characters**, such as subscripts and superscripts (like ² and ₁), that are defined as digits. It returns **False** otherwise (if the string is empty or contains non-digit characters).

LONG ANSWER TYPE QUESTIONS:

1. Write syntax for if-else and Write a Python program to find the largest of two numbers using if-else.

Ans:

The simple **if-else** statement is the most basic form of conditional logic in Python. It allows your program to choose between two outcomes: executing one block of code if a condition is true, or executing a different block of code if the condition is false.

Syntax:

```
if <condition_is_true>:
```

```
    # Code Block 1: Executes only if the condition is True.  
    statement_1
```

```
else:
```

```
    # Code Block 2: Executes only if the condition is False.  
    statement_2
```

Program to find the largest of two numbers using if-else:

Program:



```
a = int(input("Enter first number: "))
b = int(input("Enter second number: "))
if (a>b):
    print(a, f"is the largest number than {b}")
else:
    print(b, f"is the largest number than {a}")
```

Output:

```
Enter first number: 12
Enter second number: 13
13 is the largest number than 12
```

```
Enter first number: 4
Enter second number: 3
4 is the largest number than 3
```

✓ **2. Explain string counting and searching methods with examples.**

Ans:

String counting method:

It is possible to find out how many times a string is repeated in another string. This is done using the count() operation.

for example,

```
my_string = 'Count the number of spaces'
print("my_string.count(' '):", my_string.count(' '))
```

```
my_string.count(' '): 4
```

Searching methods (finding sub strings):

You can find out if one string is a substring of another string using the find() method. This method takes a second string as a parameter and checks to see if that string is in the string receiving the find() method, for example:

string.find(string_to_find)



The method returns -1 if the string is not present.

```
my_string = 'Count the number of spaces'  
print(my_string.find('zero'))
```

-1

Otherwise, it returns an index indicating the start of the substring.

For example,

```
my_string = 'Count the number of spaces'  
print(my_string.find('the'))
```

6

This prints out the value 6 (the index of the first letter of the substring “the” note strings are indexed from Zero; thus the first letter is at position Zero, the second at position one etc.

3/ Write a python program to compare three numbers using elif statement.

Ans:

```
a = int(input("Enter first number: "))  
b = int(input("Enter second number: "))  
c = int(input("Enter third number: "))  
  
# Comparing the numbers  
if a > b and a > c:  
    print(a, f"is the largest number than {b} and {c}")  
elif b > a and b > c:  
    print(b, f"is the largest number than {a} and {c}")  
elif c > a and c > b:  
    print(c, f"is the largest number than {a} and {b}")  
else:  
    print("Two or more numbers are equal and largest.")
```



Output:

```
Enter first number: 10
Enter second number: 9
Enter third number: 8
10 is the largest number than 9 and 8
```

```
Enter first number: 10
Enter second number: 10
Enter third number: 5
Two or more numbers are equal and largest.
```

4. Write a program to print factorial of a given number using recursive function.

Ans:

Recursion is a process where a function calls itself to solve a problem. A recursive function breaks down a complex problem into smaller, simpler problems. It continues calling itself until it reaches a condition that stops the recursion.

Program:

```
def factorial(n):
    if n == 0 or n == 1:
        return 1
    return n * factorial(n - 1)

n = int(input("Enter a number: "))
if n < 0:
    print("Factorial is not defined for negative numbers.")
else:
    print(f"Factorial of {n} is {factorial(n)}")
```

Output:

```
Enter a number: 6
Factorial of 6 is 720
```



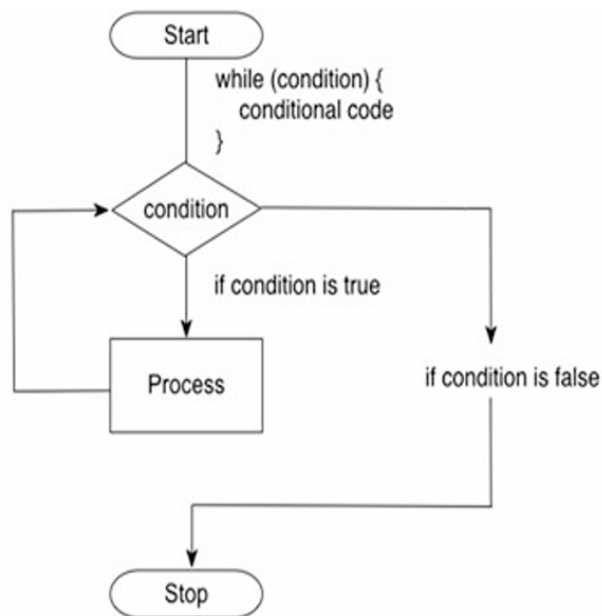
Enter a number: -3

Factorial is not defined for negative numbers.

- 5/ Explain briefly about while loop and write a program to print the sum of first 10 natural numbers using a while loop.

Ans:

- The **while** loop exists in almost all programming languages and is used to **iterative** (or repeat) one or more code statements as long as the test condition (expression) is **True**.
- This iteration construct is usually used when the number of times we need to repeat the block of code to execute is **not known**.



- The Python while loop has the basic form:

Syntax: **while** <test-condition-is-true>:
statement or statements

Program to print the sum of first 10 natural numbers using a while loop:

Program:



```
n=10
i=1
sum=0
while(i<=n):
    sum=sum+i
    i+=1
print(sum)
```

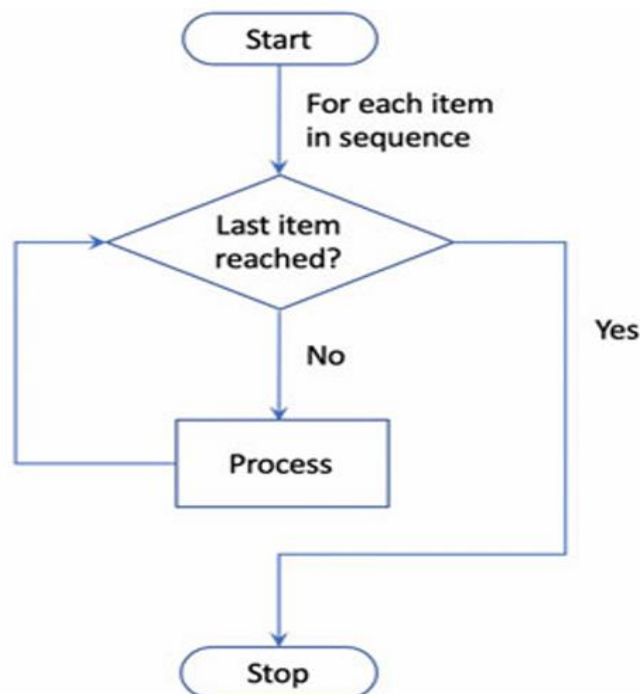
Output:

55

6 ✓ **Discuss in detail about for loop and Write a program to check whether the given number is prime number or not using for loop.**

Ans:

The for loop is used to step a variable through a series of values until a given test is met.





- In Python the approach is slightly different as the values from 0 up to 10 are represented by a **range**.
- The format of the Python for loop when using a range of values is.

Syntax: **for <variable-name> in range(...):**
 statement
 statement

Program to check whether the given number is prime number or not using for loop.

Program:

```
n= int(input("enter a number: "))
for i in range(2,n):
    if (n%i==0):
        print("The given number is not prime")
        break
else:
    print("The given number is prime")
```

Output:

```
enter a number: 7
The given number is prime

enter a number: 6
The given number is not prime
```

✓ **7/ Write a Python program to print numbers from 0 to 4 but skip number 3 using loop control statements.**

Ans: Program:



```
for i in range(5):  
    if i == 3:  
        continue  
    print(i)
```

0
1
2
4

Explanation:

- The **for** loop iterates over numbers 0 through 4.
- When i equals 3, the continue statement causes the loop to skip printing 3 and move to the next iteration.
- All other numbers (0, 1, 2, 4) are printed.

8. Write a Python program to check whether the given number is Palindrome or not.

Ans:

Program:



```
n=int(input("enter a number"))
i=n
sum=0
while n>0:
    a=n%10
    sum=sum*10+a
    n=n//10
if (sum==i):
    print("The given number is palindrome")
else:
    print("The given number is not a palindrome")
```

Output:

```
enter a number 121
The given number is palindrome

enter a number 123
The given number is not a palindrome
```

9. Explain different string case conversion methods with examples.

Ans:

String Case Conversion Methods:

These methods all return a new string; they do not modify the original string (since strings are immutable in Python).

1. upper()

- **upper()** converts all characters in a string to uppercase.
- **Example:**

```
text = "hello world"
print(text.upper())
```

Output: "HELLO WORLD"



2. lower()

- **lower()** converts all characters to lowercase.

- **Example:**

```
text = "HELLO WORLD"  
print(text.lower())
```

Output: "hello world"

3. capitalize()

- **capitalize()** converts the first character to uppercase and the rest to lowercase.

- **Example:**

```
text = "python programming"  
print(text.capitalize())
```

Output: "Python programming"

4. title()

- **title()** converts the first character of each word to uppercase and the rest to lowercase.

- **Example:**

```
text = "welcome to python"  
print(text.title())
```

Output: "Welcome To Python"

5. swapcase()

- **swapcase()** swaps the case of each character (uppercase becomes lowercase and vice versa).

- **Example:**

```
text = "Hello World"  
print(text.swapcase())
```

Output: "hELLO wORLD"

6. casefold()

- **casefold()** converts string into lowercase, similar to lower(), but handles special Unicode characters.



- **Example:**

```
text = "S"  
print(text.lower()) #  
print(text.casefold()) #
```

Output: "S"

"ss"

10. Explain Recursive function with an example.

Ans:

- A recursive algorithm is a method in programming where a function calls itself to solve a smaller version of the same problem.
- This process continues until the problem becomes simple enough to be solved directly, without any further calls.
- The key idea in recursion is that the problem is broken down into smaller, more manageable pieces, which makes it easier to solve.

Components of Recursive Algorithm

A recursive algorithm is made up of three main components: the base case, the recursive case, and the recurrence relation.

1. Base Case

The base case is the condition that stops the recursion. The base case prevents the algorithm from calling itself infinitely and eventually allows the recursion to unwind.

2. Recursive Case

The recursive case is where the function calls itself with a smaller or simpler version of the original problem. This part of the algorithm is responsible for breaking down the problem into smaller pieces that are easier to solve.

3. Recurrence Relation

The recurrence relation is the mathematical formula or expression that defines how the original problem is split into smaller subproblems.

Syntax:



```
def recurse():  
    ...  
    recurse()  
    ...  
  
recurse()
```

Diagram illustrating a recursive call: A blue box highlights the function definition and its call. An arrow points from the `recurse()` call inside the function to the `def recurse():` line, labeled "recursive call".

Example:

Fibonacci Sequence

The Fibonacci sequence is a series of numbers where each number is the sum of the two preceding ones, usually starting with 0 and 1. The sequence is defined recursively because each Fibonacci number depends on the sum of the previous two.

```
def fibonacci(n):  
    # Base cases  
    if n == 0:  
        return 0  
    elif n == 1:  
        return 1  
    # Recursive case  
    else:  
        return fibonacci(n - 1) + fibonacci(n - 2)  
  
# Example: Print first 18 Fibonacci numbers  
for i in range(8):  
    print(fibonacci(i), end=" ")
```

0 1 1 2 3 5 8 13



MODULE-III: PYTHON DATA STRUCTURES

SHORT ANSWER TYPE QUESTIONS:

✓ 1. Define list in Python with example.

A list in Python is an ordered collection of items that can store multiple values in a single variable. Lists are mutable, meaning their elements can be changed after creation. Lists are written using square brackets [].

Syntax: Lists are created using square brackets [] to enclose elements, separated by commas.

For example:

```
list1 = ['John', 'Paul', 'George', 'Ringo']  
print(list1)
```

Output:

```
['John', 'Paul', 'George', 'Ringo']
```

✓ 2. Give an example for negative indexing in lists.

Ans: Negative Indexing: Access items from the end of the list using negative indices. - 1 refers to the last item.

Example:

```
list1 = [23,'xyz',0.895, "ABC"]  
print('list1[-1]:', list1[-1])
```

Output: list1[-1]: ABC

✓ 3. What is extended slicing in Python?

Ans: Extended slicing in Python is a feature that allows sequences (like lists, tuples, and strings) to be accessed and manipulated using an additional parameter **the step value** beyond the traditional start and stop indices.

Syntax for extended slicing: sequence [start:stop:step]

✓ 4. Define tuple in python.

Ans: Tuples are an immutable ordered collection of objects; that is each element in a tuple has a specific position (its index) and that position does not change over time.



Indeed, it is not possible to add or remove elements from the tuple once it has been created.

Usage:

Tuples are commonly used when:

- You want to store a fixed set of values
- You want to ensure data cannot be changed (immutability)
- You need to return multiple values from a function

5. What is a set? Write one example.

Ans: A Set is an unordered (un-indexed) collection of immutable objects that does not allow duplicates.

Example:

```
basket = {'apple', 'orange', 'apple', 'pear', 'orange', 'banana'}  
print(basket)
```

Output:

```
{'orange', 'apple', 'pear', 'banana'}
```

6. Write the difference between remove() and discard() methods in set.

Ans: The **remove()** method removes a specified element from a set. If the element is not found, it raises a **KeyError**.

Example:

Example1:	Example2:
<pre>s = {1, 2, 3} s.remove(2) print(s)</pre>	<pre>s = {1, 2, 3} s.remove(4) print(s)</pre>
Output: {1, 3}	Output: KeyError: 4

The **discard()** method also removes a specified element but does nothing if the element is not found. No error is raised.

Example:

```
s = {1, 2, 3}  
s.discard(2)
```



```
print(s)
s.discard(4)
print(s)
```

Output:

```
{1, 3}
{1, 3}
```

7. What is a dictionary in Python? Give one simple example.

Ans: A dictionary contains a collection of indices, which are called keys, and a collection of values. Each key is associated with a single value. The association of a key and a value is called a key-value pair or sometimes an item.

Example:

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
print(my_dict)
```

Output: {'apple': 3, 'banana': 5, 'cherry': 5}

8. Write an example for nested comprehension

Nested Comprehensions: Build complex structures like matrices.. For example, a multiplication table:

```
multiplication = [[i * j for j in range(1, 6)] for i in range(2, 5)]
print(multiplication)
```

Output: [[2, 4, 6, 8, 10], [3, 6, 9, 12, 15], [4, 8, 12, 16, 20]]

MODULE-III

LONG ANSWER TYPE QUESTIONS:

1. Explain how to create a list and a list of lists in Python with examples.

Ans: A **list** in Python is an ordered, mutable collection of items enclosed in square brackets [], with elements separated by commas. Lists can hold items of any type, including numbers, strings, or even mixed types.



```
# List of integers
numbers = [1, 2, 3, 4, 5]

# List of strings
fruits = ['apple', 'banana', 'cherry']

# List with mixed types
mixed = [1, 'hello', 3.14, True]

print(numbers)
print(fruits)
print(mixed)

[1, 2, 3, 4, 5]
['apple', 'banana', 'cherry']
[1, 'hello', 3.14, True]
```

Creating a List of Lists in Python

A *list of lists* is simply a list where each element is itself a list (i.e., nested lists).

```
# List of lists (matrix)
matrix = [
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
]

print(matrix)

# Access an element, e.g., row 1, column 2
print(matrix[1][2]) # Output: 6

[[1, 2, 3], [4, 5, 6], [7, 8, 9]]
6
```

2. Explain briefly about Sorting and Merging lists with examples.



Ans: Sorting and Merging Lists:

SORTING LISTS:

▪ **sort() method:**

The **sort()** method sorts a list in-place in ascending order by default. This method changes the original list.

Example:

```
X = [54, 12, 85, 65, 74, 90]
X.sort()
print("Sorted list:",X)
```

Output:

Sorted list: [12, 54, 65, 74, 85, 90]

▪ **reverse() method:**

To sort a list in descending order, set the **reverse parameter of sort() to True**.

Example:

```
X = [54, 12, 85, 65, 74, 90]
X.sort(reverse=True)
print("sorted list:",X)
```

Output:

sorted list: [90, 85, 74, 65, 54, 12]

▪ **sorted() method:**

sorted() returns a new sorted list, leaving the original list unchanged. It is ideal when you need a temporary sort.

Example:

```
X = [54, 12, 85, 65, 74, 90]
Y=sorted(X)
print("original list",X)
print("sorted list:",Y)
```

Output:

original list [54, 12, 85, 65, 74, 90]
sorted list: [12, 54, 65, 74, 85, 90]



▪ Custom Sorting with key Parameter

You can define custom sort criteria using the key parameter, which accepts a function to customize the sort order.

Example:

```
fruit = ["banana", "apple", "kiwi"]  
fruit.sort(key=len)  
print(fruit)
```

Output:

```
['kiwi', 'apple', 'banana']
```

MERGING LISTS:

Using + operator:

It is possible to concatenate two lists together using the concatenation operator '+'.

Example:

```
data1=['dog', 'cat', 'fish']  
data2=[1,2,3,4]  
data3=data1+data2  
print(data3)
```

Output:

```
['dog', 'cat', 'fish', 1, 2, 3, 4]
```

Using extend():

Another common method is to use the **extend()** function, which modifies the original list by adding elements from another list.

Example:

```
data1=['dog', 'cat', 'fish']  
data2=[1,2,3,4]  
data1.extend(data2)  
print(data1)
```

Output:

```
['dog', 'cat', 'fish', 1, 2, 3, 4]
```



3. Explain briefly about any 5 tuple operations with an example.

Ans:

OPERATIONS ON TUPLES:

▪ **Accessing elements of a tuple:**

The elements of a Tuple can be accessed using an index in square brackets. The index returns the object at that position.

Example:

```
Tup1 = (1, 2, 3)
print('Tup1[0]:', Tup1[0])
print('Tup1[1]:\t', Tup1[1])
```

Output:

```
Tup1[0]: 1
Tup1[1]: 2
```

▪ **Negative Indexing:**

You can also use negative indices to access elements from the end of the tuple.

Example:

```
Tup1 = (1, 'abc', 2.45, 5)
print(Tup1[-2])
```

Output:

```
2.45
```

▪ **Slicing:**

Slicing is done by tuple(iterable) providing the start and end indexes for the slice, separated by a colon, within the index square brackets.

➤ **For example:**

```
Tup1 = (1, 'abc', 2.45, 5)
print(Tup1[0:3])
print(Tup1[1:])
print(Tup1[:2])
```

Output:

```
(1, 'abc', 2.45)
```



('abc', 2.45, 5)

(1, 'abc')

▪ **Reversing a Tuple:**

You can reverse a Tuple using the `:: - 1` notation (again this returns a new Tuple and has no effect on the original Tuple):

Example:

```
Tup1 = (1, 'abc', 2.45, 5)
print(Tup1[::-1])
```

Output:

(5, 2.45, 'abc', 1)

▪ **Concatenation:**

You can combine two or more tuples using the `+` operator. This operation creates a new tuple.

Example:

```
Tup1=(3,6,8,2)
Tup2=('xyz',3.2,1)
Tup3=Tup1+Tup2
print(Tup3)
```

Output:

(3, 6, 8, 2, 'xyz', 3.2, 1)

▪ **Repetition:**

The `*` operator can be used to repeat a tuple a specified number of times, also creating a new tuple.

Example:

```
Tup=(3,6,8,2)
print(Tup*2)
```

Output:

(3, 6, 8, 2, 3, 6, 8, 2)



▪ **Comparison:**

If all the elements in both the tuples are same then it returns True, otherwise it returns False.

Example:

```
Tup1=(3,6,8,2)
Tup2=('xyz',3,1)
print(Tup1==Tup2)
print(Tup1!=Tup2)
```

Output:

```
False
True
```

4. Discuss about any five tuple built- in functions in python with example.

Ans:

- **count():** Returns the number of times a specified value appears in the tuple.

Example:

```
num = (10, 20, 5, 30, 15, 5)
print("Count of 5 is ",num.count(5))
```

Output:

```
Count of 5 is 2
```

- **index():** Searches the tuple for a specified value and returns the index of its first occurrence. If the value is not found, it raises a ValueError.

Example:

```
num = (10, 20, 5, 30, 15, 5)
print("Index of 5 is ",num.index(5))
```

Output:

```
Index of 5 is 2
```

- **len():** Returns the number of elements in the tuple.

Example:

```
numbers = (10, 20, 5, 30, 15)
print("Length:",len(numbers))
```



Output:

Length: 5

- **max()**: Returns the largest item in the tuple (if all items are of the same comparable type).

Example:

```
numbers = (10, 20, 5, 30, 15)
```

```
print("Maximum:" ,max(numbers))
```

Output:

Maximum: 30

- **min()**: Returns the smallest item in the tuple (if all items are of the same comparable type).

Example:

```
numbers = (10, 20, 5, 30, 15)
```

```
print("Minimum:", min(numbers))
```

Output:

Minimum: 5

- **sum()**: Returns the sum of all items in the tuple (if they are all numbers).

Example:

```
numbers = (10, 20, 5, 30, 15)
```

```
print("Sum:",sum(numbers))
```

Output:

Sum: 80

5. Explain briefly about any 5 tuple operations with an example.

Ans:

▪ **Accessing elements of a tuple:**

The elements of a Tuple can be accessed using an index in square brackets. The index returns the object at that position.

Example:

```
Tup1 = (1, 2, 3)
```

```
print('Tup1[0]:', Tup1[0])
```



```
print('Tup1[1]:\t', Tup1[1])
```

Output:

```
Tup1[0]: 1  
Tup1[1]: 2
```

▪ **Negative Indexing:**

You can also use negative indices to access elements from the end of the tuple.

Example:

```
Tup1 = (1, 'abc', 2.45, 5)  
print(Tup1[-2])
```

Output:

```
2.45
```

▪ **Slicing:**

Slicing is done by tuple(iterable) providing the start and end indexes for the slice, separated by a colon, within the index square brackets.

➤ **For example:**

```
Tup1 = (1, 'abc', 2.45, 5)  
print(Tup1[0:3])  
print(Tup1[1:])  
print(Tup1[:2])
```

Output:

```
(1, 'abc', 2.45)  
(1, 'abc', 2.45, 5)  
(1, 'abc')
```

▪ **Concatenation:**

You can combine two or more tuples using the + operator. This operation creates a new tuple.

Example:

```
Tup1=(3,6,8,2)  
Tup2=('xyz',3.2,1)
```



```
Tup3=Tup1+Tup2
```

```
print(Tup3)
```

Output:

```
(3, 6, 8, 2, 'xyz', 3.2, 1)
```

▪ **Repetition:**

The * operator can be used to repeat a tuple a specified number of times, also creating a new tuple.

Example:

```
Tup=(3,6,8,2)
```

```
print(Tup*2)
```

Output:

```
(3, 6, 8, 2, 3, 6, 8, 2)
```

▪ **Membership:**

The in and not in keywords check if an element is present in a tuple then it returns True otherwise it returns False.

Example:

```
Tup=(3,6,8,2)
```

```
print(6 in Tup)
```

```
print(8 not in Tup)
```

Output:

```
True
```

```
False
```

▪ **Comparision:**

If all the elements in both the tuples are same then it returns True, otherwise it returns False.

Example:

```
Tup1=(3,6,8,2)
```

```
Tup2=('xyz',3,1)
```

```
print(Tup1==Tup2)
```

```
print(Tup1!=Tup2)
```



Output:

False

True

Explain in detail about any 5 set methods with examples.

- **add():**

The **add()** method inserts a single element into a set. If the element is already in the set, the set remains unchanged.

Example:

```
s1 = {'apple', 'orange', 'banana'}  
s1.add('grape')  
print(s1)
```

Output:

{'banana', 'apple', 'grape', 'orange'}

- **update():**

The **update()** method adds multiple elements to a set from an iterable (like a list, tuple, or another set).

Example1:	Example2:
<pre>s1 = {'apple', 'orange', 'banana'} s1.update(['grape']) print(s1)</pre> Output: {'banana', 'apple', 'grape', 'orange'}	<pre>s = {1, 2, 3} s.update([4, 5, 6]) print(s)</pre> Output: {1, 2, 3, 4, 5, 6}

- **remove() and discard()**

- The **remove()** method removes a specified element from a set. If the element is not found, it raises a **KeyError**.

Example1:	Example2:
<pre>s = {1, 2, 3} s.remove(2) print(s)</pre>	<pre>s = {1, 2, 3} s.remove(4) print(s)</pre>



Output:

{1, 3}

Output:

KeyError: 4

- The **discard()** method also removes a specified element but does nothing if the element is not found. **No error is raised.**

Example:

```
s = {1, 2, 3}
s.discard(2)
print(s)
s.discard(4)
print(s)
```

Output:

```
{1, 3}
{1, 3}
```

- **pop():**

The **pop() method** removes and returns an arbitrary element from the set. Since sets are unordered, you can't predict which element will be removed. If the set is empty, it raises a **KeyError**.

Example:

```
s = {1, 2, 3}
removed_element = s.pop()
print(removed_element)
print(s)
```

Output:

```
1
{2, 3}
```

- **clear():** Removes all elements from the set, making it empty.

Example:

```
s = {1, 2, 3}
s.clear()
```



```
print(s)
```

Output:

```
set()
```

Subset and Superset Methods

These methods are used to check the relationship between two sets:

- **issubset()**: Returns True if all elements of the original set are present in the set passed as an argument. The $\leq$ operator can also be used.

Example:

```
A = {1, 2}
B = {1, 2, 3, 4}
print(A.issubset(B))
print(A<=B)
```

Output:

```
True
True
```

- **issuperset()**: Returns True if all elements of the set passed as an argument are present in the original set. The $\geq$ operator can also be used.

Example:

```
A = {1, 2}
B = {1, 2, 3, 4}
print(A.issuperset(B))
print(A>=B)
```

Output:

```
False
False
```

- **isdisjoint()**: Returns True if the original set has **no** elements in common with the set passed as an argument.

Example:

```
A = {1,2,6}
B = {3,4,5}
```



```
C = {2,5,4}
print(A.isdisjoint(B))
print(A.isdisjoint(C))
```

Output:

```
True
False
```

✓ **6. Explain set operations with examples.**

Ans:

The Set container also supports set like operations such as union (|), intersection (&), difference (-) and symmetric difference (^). These are based on simple Set theory.

➤ **UNION (|):**

The Union of two sets represents the combination of all the values in the two sets.

Example:

```
s1 = {'apple', 'orange', 'banana'}
s2 = {'grapefruit', 'lime', 'banana'}
print('Union:', s1 | s2)
```

Output:

```
Union: {'orange', 'apple', 'lime', 'banana', 'grapefruit'}
```

➤ **INTERSECTION (&):**

The intersection of two sets represents the common values between two sets.

Example:

```
s1 = {'apple', 'orange', 'banana'}
s2 = {'grapefruit', 'lime', 'banana'}
print('Intersection:', s1 & s2)
```

Output:

```
Intersection: {'banana'}
```

➤ **DIFFERENCE (-):**

The difference between two sets is the set of values in the first set that are not in the second set.

Example:



```
s1 = {'apple', 'orange', 'banana'}  
s2 = {'grapefruit', 'lime', 'banana'}  
print('Difference:', s1 - s2)
```

Output:

Difference: {'orange', 'apple'}

➤ **SYMMETRIC DIFFERENCE (^):**

The symmetric difference represents all the unique values in the two sets (that is it is the inverse of the intersection).

Example:

```
s1 = {'apple', 'orange', 'banana'}  
s2 = {'grapefruit', 'lime', 'banana'}  
print('Symmetric Difference:', s1 ^ s2)
```

Output:

Symmetric Difference: {'lime', 'grapefruit', 'orange', 'apple'}

7. Explain dictionary operations with examples.

Ans:

A dictionary contains a collection of indices, which are called **keys**, and a collection of values. Each key is associated with a single value. The association of a key and a value is called a **key-value pair** or sometimes an **item**.

Using the update() Method:

- The update() method adds new key-value pairs and overwrites existing ones if the keys already exist.

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}  
my_dict.update({'apple': 10})  
print(my_dict)
```

Output: {'apple': 10, 'banana': 5, 'cherry': 5}

Assigning directly:

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}  
my_dict['apple']=10  
print(my_dict)
```



Output: {'apple': 10, 'banana': 5, 'cherry': 5}

Removing an Entry:

- An entry into the dictionary can be removed using one of the methods **pop()** or **popitem()** method or the **del** keyword.

Using pop() method:

- The pop() method removes the entry with the specified key. **This method returns the value of the key being deleted.**
- If the key is not present error will be generated.

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
my_dict.pop('apple')
```

Output: 3

```
print(my_dict)
```

Output: {'banana': 5, 'cherry': 5}

Using popitem() method:

- The popitem() method removes the last inserted item in the dictionary.
- **The key:value pair being deleted is returned from the method.**

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
my_dict.popitem()
```

Output: ('cherry', 5)

Using del keyword:

- del keyword removes the entry with the specified key from the dictionary.
- This keyword just deletes the item; **it does not return the associated value.**
- It is potentially more efficient than pop()

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
del my_dict['apple'] ## (it does not return the value)
print(my_dict)
```

Output: {'banana': 5, 'cherry': 5}

✓ **8. Explain briefly about accessing and manipulating key-values in python.**

Ans:

Accessing Items via Keys:



- You can access the values held in a Dictionary using their associated key. This is specified using either the square bracket ('[]') notation (where the key is within the brackets) or the get() method:

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
print(my_dict['apple'])
```

Output: 3

Using get():

```
my_dict.get('banana')
```

Output:5

Adding a New Entry:

- A new entry can be added to a dictionary by providing the key in square brackets and the new value to be assigned to that key:

```
my_dict = {'apple': 10, 'banana': 5, 'cherry': 5}
my_dict ['Guava']=15
print(my_dict)
```

Output: {'apple': 10, 'banana': 5, 'cherry': 5, 'Guava': 15}

Changing a Keys Value:

- The value associated with a key can be changed by reassigning a new value using the square bracket notation.

```
my_dict = {'apple': 3, 'banana': 5, 'cherry': 5}
my_dict['apple']=100
print(my_dict)
```

Output: {'apple': 100, 'banana': 5, 'cherry': 5}

9. Explain in detail about List comprehensions in python.

Ans:

List Comprehensions:

- It is a very powerful mechanism that can be used to generate new lists.

Syntax:

- Syntax for List Comprehension is



[<expression> **for** item **in** iterable <**if** optional_condition>]

Note: Whole **for** statement and expression is surrounded by the square brackets normally used to create a **List**.

- When a List Comprehension is executed, it generates a new list by applying the expression to the items in another collection.
- For example, given one list of integers(list1), another list(list2) can be created using the **List Comprehension format**:

```
list1 = [1, 2, 3, 4, 5,6]
print('list1:', list1)
list2 = [item for item in list1]
print('list2:', list2)
```

Output: list1: [1, 2, 3, 4, 5, 6]

list2: [1, 2, 3, 4, 5, 6]

- This feature is not limited to processing values in a list; any iterable collection can be used such as **Tuples** or **Sets** as the source of the values to process.
- Another feature of the List Comprehension is the ability to filter the values passed to the expression using the **optional if condition**.
- For example, if we wish to only consider even numbers for the expression then we can use the optional if statement to filter out all odd numbers:

```
list3 = [item + 1 for item in list1 if item % 2 == 0]
print('list3:', list3)
```

Output: list3: [3, 5, 7]

- Thus only the even numbers were passed to the expression item + 1 resulting in only the values 3, 5 and 7 being generated for the new list.

10. Discuss the various applications of List Comprehension in python.

Applications of List Comprehension:

- List comprehension is a programming construct used to efficiently create new lists by transforming, filtering, or combining items from existing iterables in a concise, readable way.



1. **Filtering Elements:** Extract items from a list that meet certain criteria. For example, creating a list of fruits containing the letter "a".

```
fruits = ["apple", "banana", "cherry", "kiwi", "mango"]  
newlist = [x for x in fruits if "a" in x]  
print(newlist)
```

Output: ['apple', 'banana', 'mango']

2. **Transforming Data:** Apply an operation or function to each element. For instance, squaring numbers.

```
numbers = [2, 3, 4, 5]  
squares = [n ** 2 for n in numbers]  
print(squares)
```

Output: [4, 9, 16, 25]

3. **Nested Comprehensions:** Build complex structures like matrices.. For example, a multiplication table:

```
multiplication = [[i * j for j in range(1, 6)] for i in range(2, 5)]  
print(multiplication)
```

Output: [[2, 4, 6, 8, 10], [3, 6, 9, 12, 15], [4, 8, 12, 16, 20]]

4. **Combining Iterables:** Generate coordinate pairs or combinations, such as

```
coordinates = [(x, y) for x in range(3) for y in range(3)]  
print(coordinates)
```

Output: [(0, 0), (0, 1), (0, 2), (1, 0), (1, 1), (1, 2), (2, 0), (2, 1), (2, 2)]



MODULE-IV: INTRODUCTION TO OBJECT ORIENTED PROGRAMMING

SHORT ANSWER TYPE QUESTIONS:

1/ Define a class in Python.

Ans: A class is a collection of objects. Classes are blueprints for creating objects. A class defines a set of attributes and methods that the created objects (instances) can have.

2/ What is the use of constructors in OOP?

Ans: In Object-Oriented Programming (OOP), a **constructor** is a special method used to initialize objects when they are created.

3/ What is inheritance?

Ans: Inheritance is used to inherit the properties and behaviours of one class to another. The class that inherits another class is called a **child class** and the class that gets inherited is called a **base class or parent class**.

4/ Define abstraction.

Ans: Abstraction is a process of handling complexity by hiding unnecessary details from the user and providing only necessary features or functionality.

5/ What is encapsulation?

Ans: Encapsulation in Python means binding variables and methods together within a class to restrict direct access to data. Python encapsulation helps make programs more secure, modular, and easier to manage.

Need of Encapsulation:

- Encapsulation in Python keeps internal data safe from accidental changes.
- It allows controlled access through getter and setter methods.
- It improves code readability and maintainability.
- It helps us create reusable and flexible class structures.

6/ What is polymorphism?



Ans: The word "polymorphism" means "many forms", and in programming it refers to methods/functions/operators with the same name that can be executed on many objects or classes.

7. Define getters and setters.

Ans:

Getters: A getter is a method used to read or get the value of a private variable.
Setters: A setter is a method used to change or set the value of a private variable

8. Define method overriding.

Ans:

Method overriding allows a subclass to change or customize the behavior of a method inherited from the parent class.

MODULE-IV

LONG ANSWER TYPE QUESTIONS:

1. Explain any two OOP concepts with examples.

Ans:

ENCAPSULATION:



Encapsulation in Python means binding variables and methods together within a class to restrict direct access to data. Python encapsulation helps make programs more secure, modular, and easier to manage.

Need of Encapsulation:

- Encapsulation in Python keeps internal data safe from accidental changes.
- It allows controlled access through getter and setter methods.
- It improves code readability and maintainability.
- It helps us create reusable and flexible class structures.

Access Modifiers:

In Python, encapsulation is achieved through the use of access modifiers. These access modifiers control the visibility of class attributes and methods from outside the class. The three common access modifiers are:

- **Public:** Members are accessible from outside the class.
- **Protected:** Members are accessible within the class and its subclasses.
- **Private:** Members are only accessible within the class.

1. Public Members in Class

- Public members in a class are attributes or methods that can be accessed from anywhere outside the class.
- In the following program, we have a Student class with two attributes: name and age. These attributes can be accessed from outside the class. Therefore name and age attributes in this example are public members.

Program:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student = Student("Green", 20)
print(student.name) # Accessing public attribute
print(student.age)
```



Output:

Green
20

2. Protected Members in Class

- Protected members in a class are attributes or methods that can be only be accessed from inside the same class, and from inside sub-classes.
- In the following program, we have two classes: Animal and Dog. Dog class is a subclass of Animal. Inside the Animal class, we have an attribute `_type`, which is protected.

Example:

```
class Animal:
    def __init__(self):
        self._type = "Unknown"

class Dog(Animal):
    def __init__(self):
        super().__init__()
        self._type = "Dog"

dog = Dog()
print(dog._type)
```

Dog

3. Private Member in Class

- Private members in a class are attributes or methods that can be only be accessed from inside the same class.
- In the following program, we have a class: BankAccount. BankAccount class has an attribute `__balance`, which is a private member.

Example:



```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount()
account.deposit(50000)
print("balance:", account.get_balance()) # Accessing private method

balance: 50000
```

INHERITANCE:

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Child class is the class that inherits from another class, also called derived class.

Creating a Parent Class

The class whose attributes and methods are inherited is called as parent class. It is defined just like other classes i.e. using the class keyword.

Syntax:

```
class ParentClassName:
    {class body}
```

Creating a Child Class

Classes that inherit from base classes are declared similarly to their parent class, however, we need to provide the name of parent classes within the parentheses.



Syntax:

```
class SubClassName (ParentClass1[, ParentClass2, ...]):  
    {sub class body}
```

Example

Create a class named Dog, which will inherit the properties and methods from the Animal class:

```
class Animal:  
    name = ""  
    def eat(self):  
        print("I can eat")  
  
    # inherit from Animal  
class Dog(Animal):  
    def display(self):  
        print("My name is ", self.name)  
  
labrador = Dog()  
labrador.name = "Jimmy"  
labrador.eat()  
  
    # call subclass method  
labrador.display()
```

Output:

```
I can eat  
My name is Jimmy
```

2. Write a program to define a class Student and display details using a method.

Ans:

Program:



```
class Student:

    def __init__(self, name, roll_number, dept):

        self.name = name          # Instance attribute 1
        self.roll_number = roll_number # Instance attribute 2
        self.dept = dept          # Instance attribute 3

    def display_details(self):

        print("Student Details:")
        print(f"Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Dept: {self.dept}")
        print("-" * 30)

student1 = Student(name="John", roll_number="A101", dept="AIML")
print("Displaying Details for Student 1")
student1.display_details()

student2 = Student(name="Bob", roll_number="C205", dept="CSE")
print("\nDisplaying Details for Student 2")
student2.display_details()
```

Output:

```
Displaying Details for Student 1
Student Details:
Name: John
Roll Number: A101
Dept: AIML
-----

Displaying Details for Student 2
Student Details:
Name: Bob
Roll Number: C205
Dept: CSE
-----
```



3. Write a Python program to demonstrate constructor usage.

Ans:

To understand the meaning of classes we have to understand the built-in `__init__()` method.

All classes have a method called `__init__()`, which is always executed when the class is being initiated.

Use the `__init__()` method to assign values to object properties, or other operations that are necessary to do when the object is being created:

```
class Book:

    def __init__(self, title, author):

        self.title = title
        self.author = author
    def display_info(self):

        print("-----")
        print(f"Title: {self.title}")
        print(f"Author: {self.author}")

book1 = Book("Think Python", "Allen Downey")

book2 = Book("Learning Python ", "Mark Lutz")

print("\nDisplaying Book Information")
book1.display_info()
book2.display_info()
```

Output:

```
--- Displaying Book Information ---
-----
Title: Think Python
Author: Allen Downey
-----
Title: Learning Python
Author: Mark Lutz
```



4. Write a Python program using inheritance (Parent → Child).

Ans:

Inheritance allows us to define a class that inherits all the methods and properties from another class.

Parent class is the class being inherited from, also called base class.

Syntax:

```
class ParentClassName:  
    {class body}
```

Child class is the class that inherits from another class, also called derived class.

Syntax:

```
class SubClassName (ParentClass1[, ParentClass2, ...]):  
    {sub class body}
```

Program:

Create a class named Dog, which will inherit the properties and methods from the Animal class:

```
# Parent class  
class Animal:  
    def eat(self):  
        print("Animal eats food")  
# Child class  
class Dog(Animal):  
    def bark(self):  
        print("Dog barks")  
# Creating object of child class  
d = Dog()  
d.eat()  # inherited from parent  
d.bark() # child class method
```



Output:

```
Animal name: Alex  
Alex always barks
```

5. Explain method overriding with a suitable example.

Ans:

Runtime polymorphism means that the behavior of a method is decided while program is running, based on the object calling it.

In Python, this happens through Method Overriding a child class provides its own version of a method already defined in the parent class. Since Python is dynamic, it supports this, allowing same method call to behave differently for different object types.

Example: In this example, we create a parent and child class to show method overriding. The child class customizes the number of sides in Python for a square.

Program:

```
class Shape:
    def sides(self):
        print("This shape has many sides")

class Square(Shape):
    def sides(self):
        print("A square has 4 sides")

# Creating objects
s1 = Shape()
s2 = Square()

# Calling methods
s1.sides()    # (Parent class method)
s2.sides()    # (Child class overrides the method)
```

Output:

```
This shape has many sides  
A square has 4 sides
```



6. Explain the difference between method overloading and overriding.

Ans:

Method Overloading	Method Overriding
Occurs within a single class	Occurs between a parent and child class
Involves multiple methods with the same name but different parameters	Involves a method in the child class with the same name and parameters as a method in the parent class
Decision is made at compile-time (static binding)	Decision is made at runtime (dynamic binding)
The return type can be the same or different	The return type must be the same or a subtype in the child class
No relationship required between the overloaded methods	Requires an inheritance relationship between the parent and child classes
Overloaded methods can have different access modifiers	Overriding methods must have the same or higher access modifier in the child class
Overloading allows methods with different functionality to have the same name	Overriding allows child classes to provide specific implementations for inherited methods
Method parameters can vary in type, order, or number	Method parameters must have the same type, order, and number in both parent and child classes
Method overloading is achieved within a single class by changing the method signature	Method overriding is achieved by using the <code>@Override</code> annotation in the child class
Overloading is a form of compile-time polymorphism	Overriding is a form of runtime polymorphism



7. Implement bank account class by using constructor.

Ans:

Program:

```
class BankAccount:
```

```
    # Constructor to initialize account holder and balance
```

```
    def __init__(self, account_holder, initial_balance=0):
```

```
        self.account_holder = account_holder
```

```
        # Ensure balance is not negative
```

```
        if initial_balance >= 0:
```

```
            self.balance = initial_balance
```

```
        else:
```

```
            print("Warning: Initial balance cannot be negative. Setting balance to 0")
```

```
            self.balance = 0
```

```
    # Method to deposit money
```

```
    def deposit(self, amount):
```

```
        if amount > 0:
```

```
            self.balance += amount
```

```
            return f'Deposit successful. New balance:{self.balance:}'
```

```
        else:
```

```
            return "Deposit amount must be positive."
```

```
    # Method to withdraw money
```

```
    def withdraw(self, amount):
```

```
        if amount > 0:
```

```
            if self.balance >= amount:
```

```
                self.balance -= amount
```

```
                return f'Withdrawal successful. New balance:{self.balance:}'
```

```
            else:
```

```
                return f'Withdrawal failed. Insufficient funds. Current balance: {self.balance:}'
```

```
        else:
```



```
return "Withdrawal amount must be positive."
```

```
# Method to check balance
```

```
def get_balance(self):
```

```
    return f'Account balance for {self.account_holder},{self.balance:}'
```

```
# String representation of the object
```

```
def __str__(self):
```

```
    return f'Account Holder: {self.account_holder}, Balance:{self.balance:}'
```

```
# Create accounts
```

```
account1 = BankAccount("Alex", 1000)
```

```
account2 = BankAccount("Jimmy",500)
```

```
print(account1)
```

```
print(account2)
```

```
# Deposit money
```

```
print(account1.deposit(250))
```

```
# Withdraw money
```

```
print(account1.withdraw(100))
```

```
# Attempt to withdraw more than balance
```

```
print(account2.withdraw(750))
```

```
# Check balances
```

```
print(account1.get_balance())
```

```
print(account2.get_balance())
```

Output:

Account Holder: Alex, Balance:1000

Account Holder: Jimmy, Balance:500

Deposit successful. New balance:1250

Withdrawal successful. New balance:1150

Withdrawal failed. Insufficient funds. Current balance: 500

Account balance for Alex,1150

Account balance for Jimmy,500

8. Create and explain abstract class with an example code.



Ans:

Program:

```
from abc import ABC, abstractmethod
```

```
# Abstract class
```

```
class Vehicle(ABC):
```

```
    def __init__(self, brand, model):
```

```
        self.brand = brand
```

```
        self.model = model
```

```
# Abstract method to start the engine
```

```
    @abstractmethod
```

```
    def start_engine(self):
```

```
        pass
```

```
# Abstract method to stop the engine
```

```
    @abstractmethod
```

```
    def stop_engine(self):
```

```
        pass
```

```
    def __str__(self):
```

```
        return f "{self.brand} {self.model}"
```

```
# Concrete subclass: Car
```

```
class Car(Vehicle):
```

```
    def start_engine(self):
```

```
        return f "{self} engine started with a key ignition."
```

```
    def stop_engine(self):
```

```
        return f "{self} engine stopped."
```

```
# Concrete subclass: ElectricScooter
```

```
class ElectricScooter(Vehicle):
```

```
    def start_engine(self):
```

```
        return f "{self} motor started silently with a button."
```



```
def stop_engine(self):  
    return f "{self} motor powered down."  
  
# Usage  
car = Car("Toyota", "Corolla")  
scooter = ElectricScooter("Xiaomi", "M365")  
  
print(car.start_engine())  
print(car.stop_engine())  
print(scooter.start_engine())  
print(scooter.stop_engine())
```

Output:

Toyota Corolla engine started with a key ignition.
Toyota Corolla engine stopped.
Xiaomi M365 motor started silently with a button.
Xiaomi M365 motor powered down.

9. Explain Encapsulation with an example code.

Ans:

Encapsulation in Python means binding variables and methods together within a class to restrict direct access to data. Python encapsulation helps make programs more secure, modular, and easier to manage.

Need of Encapsulation:

- Encapsulation in Python keeps internal data safe from accidental changes.
- It allows controlled access through getter and setter methods.
- It improves code readability and maintainability.
- It helps us create reusable and flexible class structures.

Access Modifiers:

In Python, encapsulation is achieved through the use of access modifiers. These access modifiers control the visibility of class attributes and methods from outside the class. The three common access modifiers are:

- **Public:** Members are accessible from outside the class.



- **Protected:** Members are accessible within the class and its subclasses.
- **Private:** Members are only accessible within the class.

1. Public Members in Class

- Public members in a class are attributes or methods that can be accessed from anywhere outside the class.
- In the following program, we have a Student class with two attributes: name and age. These attributes can be accessed from outside the class. Therefore name and age attributes in this example are public members.

Program:

```
class Student:
    def __init__(self, name, age):
        self.name = name
        self.age = age

student = Student("Green", 20)
print(student.name) # Accessing public attribute
print(student.age)
```

Output:

```
Green
20
```

2. Protected Members in Class

- Protected members in a class are attributes or methods that can be only be accessed from inside the same class, and from inside sub-classes.
- In the following program, we have two classes: Animal and Dog. Dog class is a subclass of Animal. Inside the Animal class, we have an attribute `_type`, which is protected.

Example:



```
class Animal:
    def __init__(self):
        self._type = "Unknown"

class Dog(Animal):
    def __init__(self):
        super().__init__()
        self._type = "Dog"

dog = Dog()
print(dog._type)
```

Dog

3. Private Member in Class

- Private members in a class are attributes or methods that can be only be accessed from inside the same class.
- In the following program, we have a class: BankAccount. BankAccount class has an attribute `__balance`, which is a private member.

Example:

```
class BankAccount:
    def __init__(self):
        self.__balance = 0

    def deposit(self, amount):
        self.__balance += amount

    def get_balance(self):
        return self.__balance

account = BankAccount()
account.deposit(50000)
print("balance:", account.get_balance()) # Accessing private method

balance: 50000
```



10. Explain types of inheritance with example codes.

Ans:

Python offers various types of inheritance based on the number of child and parent classes involved in the inheritance.

1. Single Inheritance in Python
2. Multiple Inheritance in Python
3. Multilevel Inheritance in Python
4. Hierarchical Inheritance in Python
5. Hybrid Inheritance in Python

Single Inheritance:

Single inheritance in Python means one class inherits from one parent class.

Example program:

```
class Person:
    def display_name(self):
        print("Name: Alex")

class Student(Person):
    def display_course(self):
        print("Course: Python Programming")

# Using the child class
s = Student()
s.display_name()
s.display_course()
```

Name: Alex
Course: Python Programming

Multiple Inheritance:

In multiple inheritance, a single subclass inherits from multiple superclasses. So, the child class can access attributes and methods from two or more parent classes.

However, if the parent classes have methods with the same name, the child class uses the method from the first parent in the order they are listed.



```
# Parent class 1
class Teacher:
    def teach(self):
        print("Teaches Math and CP")

# Parent class 2
class Coach:
    def train(self):
        print("Trains in Football and Athletics")

# Child class inherits from both Teacher and Coach
class Student(Teacher, Coach):
    def activities(self):
        print("Student is active in both academics and sports")
        self.teach() # Calling method from Teacher
        self.train() # Calling method from Coach

# Create object of Student
s = Student()
s.activities()
```

```
Student is active in both academics and sports
Teaches Math and CP
Trains in Football and Athletics
```

Multilevel Inheritance:

Python multilevel inheritance means a class inherits from a child class, which itself inherits from another parent class. This forms a chain of inheritance, passing features from one level to the next.

With multilevel inheritance in Python, we can build step-by-step class hierarchies, allowing each level to extend or reuse the previous one.



```
# Base class
class Grandparent:
    def show_grandparent(self):
        print("Grandparent: Shares wisdom")

# Intermediate class inheriting from Grandparent
class Parent(Grandparent):
    def show_parent(self):
        print("Parent: Provides guidance")

# Child class inheriting from Parent
class Child(Parent):
    def show_child(self):
        print("Child: Learns and grows")

# Create object of Child class
c = Child()
c.show_grandparent()
c.show_parent()
c.show_child()
```

```
Grandparent: Shares wisdom
Parent: Provides guidance
Child: Learns and grows
```

Hierarchical Inheritance:

Hierarchical inheritance is the opposite of multiple inheritance. Therefore, more than one child class can be derived from a single-parent class.



```
class Shape:
    def __init__(self, color):
        self.color = color

    def describe(self):
        return f"A shape of color {self.color}."

# Child Class 1 (inherits from Shape)
class Circle(Shape):
    def area(self, radius):
        # Area of a circle: pi * r^2
        return 3.14159 * radius * radius

# Child Class 2 (inherits from Shape)
class Square(Shape):
    def area(self, side):
        return side * side

# Usage
red_circle = Circle("Red")
blue_square = Square("Blue")

print(f"{red_circle.describe()} Area: {red_circle.area(2):.2f}")
print(f"{blue_square.describe()} Area: {blue_square.area(4)}")
```

A shape of color Red. Area: 12.57
A shape of color Blue. Area: 16

Hybrid Inheritance:

Hybrid inheritance combines two or more types of inheritance. Hence, we can see multiple relationships between parent and child classes across different levels. We can say that hybrid inheritance in Python is a mixture of more than one inheritance style, like single, multiple, or multilevel.



```
class SpaceAgency:
    def mission(self):
        print("Planning missions")

class Rocket(SpaceAgency):
    def launch(self):
        print("Launching rocket")

class Satellite(SpaceAgency):
    def orbit(self):
        print("Orbiting Earth")

class SpaceMission(Rocket, Satellite): # Hybrid inheritance
    def execute(self):
        print("Executing combined mission")

obj = SpaceMission()
obj.mission()
obj.launch()
obj.orbit()
obj.execute()

Planning missions
Launching rocket
Orbiting Earth
Executing combined mission
```



MODULE-V: FILE HANDLING AND EXCEPTION HANDLING

SHORT ANSWER TYPE QUESTIONS:

1. What is a file path?

Ans: A **file path** is the location of a file or folder in a computer's storage system. It tells the operating system where a file is stored so that Python program can find it.

2. What is append mode in file handling?

Ans: File is in write mode only; new data will be added to the end of existing data i.e. no overwriting. If file does not exist it is created.

3. What is exception handling?

Ans: Exception Handling is a mechanism in programming that allows a program to detect and handle errors during runtime without crashing

4. What is the purpose of `with open()`?

Ans: The purpose of using **with open()** in Python file handling is to automatically manage resources especially opening and closing files safely and cleanly.

Example:

Reading a file:

```
with open("data.txt", "r") as file:  
    content = file.read()  
    print(content)
```

5. What is an exception in Python?

Ans: An exception is an error that occurs while a program is running, interrupting the normal flow of execution.

6. What is a module?



Ans: A Module allows you to logically organize your Python code. When code gets to be large enough, the tendency is to break it up into organized pieces that can still interact with one another at a functioning level.

- These pieces generally have attributes that have some relation to one another.
- These pieces should be shared, so Python allows a module the ability to “bring in” and use attributes from other modules to take advantage of work that has been done, maximizing **code reusability**.

7. Discuss “w+” in brief.

Ans:

- W+ opens a file for both writing and reading.
- If the file already exists, its contents are truncated (cleared) to zero length.
- If the file does not exist, a new file is created

8. Discuss about Logical Errors.

Ans:

These are the most challenging to find because the code runs without crashing and without raising an error message. However, the program produces an incorrect or unexpected result.

Examples:

- Using the wrong mathematical formula or algorithm.
- Incorrect logic in an if statement that leads to the wrong path of execution.

MODULE-V

LONG ANSWER TYPE QUESTIONS:

1. Explain about file handling and steps involved in file handling in detail.

Ans:

File handling refers to the process of performing operations on a file, such as creating, opening, reading, writing and closing it through a programming interface. It involves managing the data flow between the program and the file system on the storage device, ensuring that data is handled safely and efficiently.

Need of File Handling



- To store data permanently, even after the program ends.
- To access external files like .txt, .csv, .json, etc.
- To process large files efficiently without using much memory.
- To automate tasks like reading configs or saving outputs.

STEPS IN DATA FILE HANDLING

1. OPENING FILE

We should first open the file for read or write by specifying the name of file and mode. File can be opened for either – read, write, append.

Syntax:

```
file_object = open(filename)
or
file_object = open("filename", "mode")
```

The default mode is “read”

Example: myfile = open(“story.txt”)

Here disk file “story.txt” is loaded in memory and its reference is linked to “myfile” object, now python program will access “story.txt” through “myfile” object. “story.txt” is present in the same folder where .py file is stored otherwise if disk file to work is in another folder we have to give full path.

2. PERFORMING READ/WRITE

Once the file is opened we can either read or write for which file is opened using various functions available.

Mode	Description
r (Read)	Opens a file for reading, error if file does not exist
w (Write)	Opens a file for Writing, If file not exists, file is created If file exists, python will truncate existing data and overwrite the file
a (Append)	Opens a file for Appending, File is in write mode only, new data will be added to the end of existing data i.e. no overwriting. If file not exists it is created



3. CLOSING FILE

After performing operation, we must close the file and release the file for other application to use it,

Syntax: `File_object.close()`

2. Explain file reading techniques: `read()`, `readline()`, `readlines()`.

Ans:

READING FROM A FILE:

Read from file python provide many functions like:

- **Reading an Entire file (`Read()`):**

We can read an entire file using the keyword `read()`.

Syntax: `Filehandle.read()`

Example:

sample.txt:

Python is an interpreted Language.

Python is a high-level, general-purpose programming language known for its clear, readable syntax.

It was created by Guido van Rossum and first released in 1991.

Program:

```
fp=open("sample.txt","r")
b=fp.read()
print(b)
fp.close()
```

Python is an interpreted Language.

Python is a high-level, general-purpose programming language known for its clear, readable syntax.

It was created by Guido van Rossum and first released in 1991.

Readline():

We can read a line of input using keyword `readline()`.

Syntax: `Filehandle.readline()`



Example:

Program:

```
fp=open("sample.txt","r")
b=fp.readline()
c=fp.readline()
print(b)
print(c)
fp.close()
```

Python is an interpreted Language.

Python is a high-level, general-purpose programming language known for its clear, readable syntax.

Readlines():

Readlines() Reads all lines and returns them in a list.

Syntax: Filehandle.readlines()

Program:

```
fp=open("sample.txt","r")
b=fp.readlines()
print(b)
fp.close()
```

```
['Python is an interpreted Language.\n', 'Python is a high-level, general-purpose\nprogramming language known for its clear, readable syntax.\n', 'It was created by\nGuido van Rossum and first released in 1991.']
```

3. Explain the different types of errors in python.

Ans:

TYPES OF ERRORS:



1. Syntax Errors

These occur when the code violates the grammatical rules of the Python language. The Python interpreter detects these during the parsing phase, and the program will not run.

Examples:

- Forgetting a colon (:) at the end of a def, if, for, or while statement.

```
if x == 10
print("x is 10")
```

```
Cell In[7], line 1
    if x == 10
        ^
SyntaxError: expected ':'
```

- Mismatched parentheses or quotes.

IndentationError: Incorrect indentation, which is significant in Python (can lead to an IndentationError or TabError).

Example:

```
if x == 10:
print("x is 10")
```

Output:

```
print("x is 10")
^
```

IndentationError: expected an indented block after 'if' statement on line 1

2. Runtime Errors (Exceptions)

These errors occur while the program is executing, even if the syntax is correct. They halt the program's execution if not handled properly. These errors cannot be detected until the code is actually run.

Examples:

ZeroDivisionError: This exception is thrown whenever a division by zero is attempted.

Example:



```
a=5  
b=0  
print(a/b)
```

ZeroDivisionError

NameError: When a variable (or) function name cannot be found in the current scope, then exception NameError is thrown.

Example:

```
a=5  
b=6  
print(a/B)
```

NameError

TypeError: When an operation (or) function is done to an object of the incorrect type, such as by adding a string to an Integer, an exception is thrown.

Example:

```
a=5  
b="abc"  
print(a/b)
```

TypeError

IndexError: This exception is thrown when a list, tuple (or) other sequence type's index is outside of borders.

Example:

```
a=[1,3,5.7]  
print(a[4])
```

IndexError

KeyError: When a key cannot be found in a dictionary, this exception is thrown.

Example:



```
a={1:"a",2:"b",3:"c"}  
print(a[4])
```

KeyError

Value error: This exception is thrown when an invalid argument or input is passed to a function. An example would be trying to convert a string to an integer when the string does not represent a valid integer.

Example:

```
a="abcde"  
print(float(a))
```

ValueError

FileNotFoundError (IOError): Trying to open a file that doesn't exist.

Example:

```
fp = open("C:/Users/User/Documents/kample.txt",'r')  
content = fp.readlines()  
for line in content:  
    print(line)
```

FileNotFoundError

Traceback (m

3. Logical Errors (Bugs)

These are the most challenging to find because the code runs without crashing and without raising an error message. However, the program produces an incorrect or unexpected result.

Examples:

- Using the wrong mathematical formula or algorithm.
- Incorrect logic in an if statement that leads to the wrong path of execution.
- Making a mistake in a mathematical calculation, such as using subtraction instead of addition.



4. Explain briefly about try-except statement with an example.

Ans:

Try -Except Statements: The try and except statements are used to handle runtime errors.

Syntax:

```
try :  
    statements  
except :  
    statements
```

The try statement works as follows:-

- First, the try clause (the statement(s) between the try and except keywords) is executed.
- If no exception occurs, the except clause is skipped and execution of the try statement is finished.
- If an exception occurs during execution of the try clause, the rest of the clause is skipped. Then if its type matches the exception named after the except keyword, the except clause is executed, and then execution continues after the try statement.

Example:

```
X=int(input("Enter the value of X"))  
Y=int(input("Enter the value of Y"))  
try:  
    result = X / ( X - Y )  
    print("result=",result)  
except ZeroDivisionError:  
    print("Division by Zero")
```



Output:

Enter the value of X:10	Enter the value of X:10
Enter the value of Y:10	Enter the value of Y:5
Division by Zero	result= 2.0

5. Explain briefly about try-except-finally statement with an example.

Ans:

Try –Except-Finally:

- A finally clause is always executed before leaving the try statement, whether an exception has occurred or not.
- The finally clause is also executed “on the way out” when any other clause of the try statement is left via a break, continue or return statement.

Syntax:

```
try:
    Statements
except:
    Statements
finally:
    Statements
```

Example:

```
X=int(input("Enter the value of X:"))
Y=int(input("Enter the value of Y:"))
try:
    result = X / ( X-Y )
except ZeroDivisionError:
    print("Division by Zero")
else:
    print("result=",result)
finally:
    print ("executing finally clause")
```



Output:

Enter the value of X:10	Enter the value of X:10
Enter the value of Y:5	Enter the value of Y:10
result= 2.0	Division by Zero
executing finally clause	executing finally clause

6. Write a program to read a text file and count words and lines.

Ans:

Text file:

```
Python is an interpreted language.  
Python is a high-level programming language.  
This is an example to create a file in python.
```

Program:

```
fp = open("C:/Users/User/Documents/sample.txt",'r')  
lines =0  
word = 0  
for line in fp:  
    words = line.split ( )  
    lines= lines+1  
    word = word + len(words)  
print("Number of line", lines)  
print("Number of words", word)
```

```
Number of line 3  
Number of words 21
```

7. Explain how to create and import modules in Python with examples.

Ans:

Creating and Managing Modules:

To create a module:

1. Write your functions, variables, or classes in a .py file. For example, create a file called module1.py:



Example:

```
def sample(name):  
    print("hello", name)
```

Importing and Using a Module

To use your module in another script, simply import it.

Example:

```
import module1  
module1.sample("John")
```

Output:

Hello John

You can also import specific items:

- It allows us to import specific attributes **from** a module into the current namespace.

➤ **Example:**

```
From module1 import sample  
Sample("John")
```

Or use an alias:

```
import module1 as jv  
jv.sample("John")
```

8. Explain how to create packages in Python with example.

- Python allows developers to organize modules together into packages, in a hierarchical structure based on directories.
- A package is defined as
 - a directory containing one or more Python source files and
 - an optional source file named `__init__.py`. This file may also contain code that is executed when a module is imported from the package

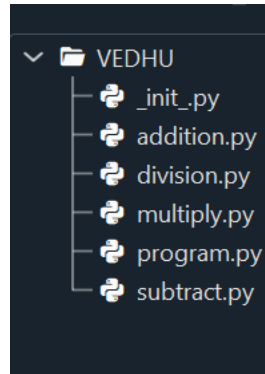
Creating, Managing, and Working with Packages

Step1: create the package directory



- For example, the following picture illustrates a package **VEDHU** containing five modules addition, division, multiply, program and subtract.

Step2: write modules for VEDHU directory add save the modules in VEDHU directory as shown in below figure



addition.py	division.py	multiply.py	subtract.py
def add(a,b): print(a+b)	def div(a,b): print(a/b)	def mul(a,b): print(a*b)	def sub(a,b): print(a-b)

Step3: add the _init_.py file in the calculator directory

- A directory must contain the file named _init_.py for python to consider it as a package

Add the following code in the _init_.py file

```
__all__ = ["addition", "subtract", "multiply",  
"division"]  
  
from . import addition  
from . import subtract  
from . import division  
from . import multiply
```

Step4: To test your package

- For testing the package **program.py** is used.
- In **program.py** actual arguments were passed as shown in below figure.

program.py



```
from subtract import sub
from multiply import mul
from division import div
add(10,5)
sub(10,5)
mul(10,5)
div(10,5)
```

Output:

```
15
5
50
2.0
```

9. Implement a program to remove white spaces in a file using rstrip.

Ans:

rstrip() method:

- The rstrip() method in Python is used to remove trailing characters from the right end of a string.
- By default, it strips whitespace such as spaces, tabs, and newline characters, but you can also specify a set of characters to remove.
- It returns a new string without altering the original one, making it especially useful for cleaning up text data—for example, when processing lines from a file that often end with unwanted spaces or newline symbols.

```
fp=open("example.txt","r")
for line in fp:
    print(line)
fp.close()
```

Python is an Interpreted language.

python is a easy to understandable language.

it is a high level language.



```
fp=open("example.txt","r")
for line in fp:
    line=line.rstrip()
    print(line)
```

Python is an Interpreted language.
python is a easy to understandable language.
it is a high level language.

```
fp=open("example.txt","r")
for line in fp:
    print(line.rstrip("high level language."))
fp.close()
```

Python is an Interpreted language.

python is a easy to understandable language.

it is

11. Explain about Raising Custom Exceptions in Python.

Ans:

- Raising custom exceptions in Python allows you to create specific error types that are more descriptive and easier to handle than the built-in exceptions.
- In Python, you can raise either built-in or custom exceptions. When you raise an exception, the result is the same as when Python does it.
- You get an exception traceback, and your program crashes unless you handle the exception on time.
- The general syntax to raise an exception is:
raise [expression [from_another_expression]]
- A raise keyword with no argument reraises the active exception. Note that you can only use a bare raise in except code blocks with an active exception. Otherwise, you'll get a [RuntimeError](#) exception:



`raise`

`RuntimeError: No active exception to reraise`

In this example, you're in a context with no active exceptions. Therefore, Python can't reraise a previous exception. Instead, it raises a RuntimeError exception.

Raise Exceptions are used:

- To stop execution when something invalid happens.
- To alert the user/programmer about incorrect input or logic.
- To provide meaningful error messages instead of silent failures.

Using raise Keyword

- The raise keyword is used to trigger an exception.
- You can raise built-in exceptions with custom messages.

```
age = -5
if age < 0:
    raise ValueError("Age cannot be negative!")
```

Output:

`ValueError: Age cannot be negative!`

Raising General Exceptions

- You can raise the base Exception directly with your own message.

```
password = "abc"
if len(password) < 6:
    raise Exception("Password must be at least 6 characters long!")
```

Output:

`Exception: Password must be at least 6 characters long!`

Handling Exceptions with try...except

- Use try...except to catch and handle raised exceptions gracefully.

```
try:
    num = 10
    den = 0
```



```
if den == 0:  
    raise Exception("Denominator cannot be zero!")  
result = num / den  
except Exception as e:  
    print("Error:", e)
```

Output:

```
Error: Denominator cannot be zero!
```

Multiple Conditions

- You can raise different exceptions depending on the situation.

```
value = "xyz"  
if not value.isdigit():  
    raise Exception("Input must be a number!")  
elif int(value) < 0:  
    raise Exception("Number must be positive!")
```

Output:

```
Exception: Input must be a number!
```